

GRAPHQL

Desarrollo de Software en Sistemas Distribuidos

GRAPHQL

Lenguaje de Consulta -
Entorno de Ejecución

<https://graphql.org>

GraphQL es un lenguaje de consulta de datos, pero también un entorno de ejecución, porque permite llevar a cabo las consultas responsabilizándose del análisis y validación de estas además de la serialización de las respuestas (es decir, convertir el objeto a devolver)

Conceptos Previos

RPC

Obtener aplicaciones que funcionan exactamente igual en un ambiente centralizado que distribuido.

- Pasaje de parámetros
- Binding
- Tratamiento de excepciones
- Transparencia de la representación de datos

SOAP

Basado en XML

- Interface de invocación a una función distribuida similar a RPC.
- describe el formato de los mensajes que utiliza y a cuales puede responder.
- Menor Flexibilidad.

RESTFULL

El más popular

- Interfaz común, operaciones GET, POST, PUT y DELETE.
- No tienen estado ni acceden a información de contexto.
- Permite transmitir XML, JSON, Text.

Problemas de REST

Overfetching

Una respuesta supera las necesidades de datos de nuestra aplicación.

Underfetching (captura ineficiente)

Varias consultas para obtener una cantidad de información necesaria

Gestión de Endpoints

A medida que cambian las necesidades del cliente, normalmente tiene que crear nuevos endpoints, y esos puntos finales pueden comenzar a multiplicarse rápidamente.

Con GraphQL, la arquitectura típica implica un único endpoint sobre el cual definimos "qué" queremos consultar y "cómo" debe ser el formato de la respuesta.

Ventajas GraphQL

Exactamente lo
que necesito

Muchos recursos en
una misma petición

Sistema
de Tipos

Otros

**Con GraphQL se
pueden restringir los
datos que deben
obtenerse del
servidor**

**Las consultas
GraphQL recuperan
los objetos
asociados, mientras
que el REST lo hará a
partir de peticiones a
varias URL.**

**GraphQL es
fuertemente tipado**

Las consultas se basan en campos y sus datos asociados a tipos. Si hay una falta de coincidencia de tipos en una consulta GraphQL, las aplicaciones del servidor devuelven un mensaje de error.

Otros

- Introspección
- Control de versiones
- Ecosistema en crecimiento

Desventajas GraphQL

Complejidad de Consultas

GraphQL no elimina los cuellos de botella de rendimiento cuando tiene que acceder a varios campos en una consulta.

Limitación de Tasa

Mientras que en REST se permiten una determinada cantidad de solicitudes por día, para Graphql esto es más difícil.

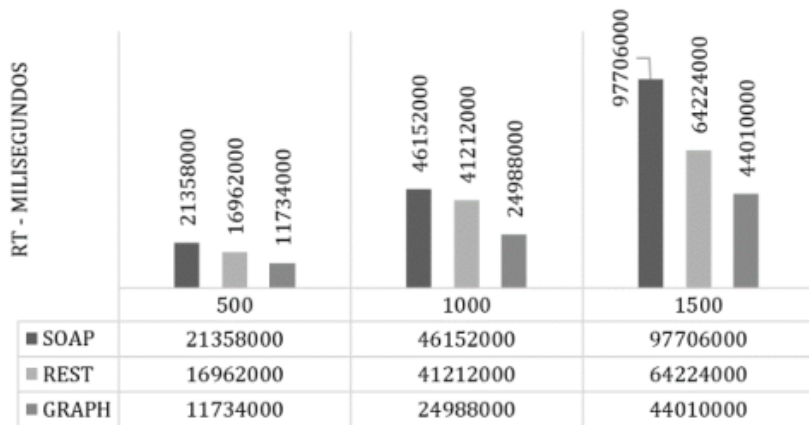
- Por lo general lo que se hace es reducir las profundidades de la consulta o su complejidad.

Cache

En REST, se accede a los recursos con URL. Almacenar en cache es mantener la URL.

- En GraphQL puede ser más complejo porque aunque se haga la consulta sobre las mismas entidades la consulta puede ser diferente.

Comparativa REST / GRAPHQL



- La tecnología REST es la más utilizada y ampliamente aceptada.

- Las respuestas de una API GraphQL son considerablemente más reducidas que las generadas con una API REST.

- Bajo estrés una API REST presenta mayor velocidad de respuesta.

- REST impone que debe utilizarse el caché mientras que GraphQL como tecnología no ofrece de forma natural este middleware. Por esto una API REST tendrá constantemente una velocidad mayor al momento de responder peticiones comunes..

- GraphQL es mucho más ventajoso en las áreas de underfetching y overfetching.

- En general, desde el punto de vista de rendimiento, GraphQL debería utilizarse cuando se requiere menor utilización de la memoria, cuando los recursos que la API maneja están estrechamente relacionados entre sí y cuando no se considere que vaya a utilizarse la data completa de los recursos

REST debería utilizarse en caso de necesitarse mayor velocidad de respuesta, cuando los recursos no están relacionados entre sí de una forma tan estrecha y cuando se vaya a utilizar toda la data de cada uno de los recursos manejados.

Principios de Diseño

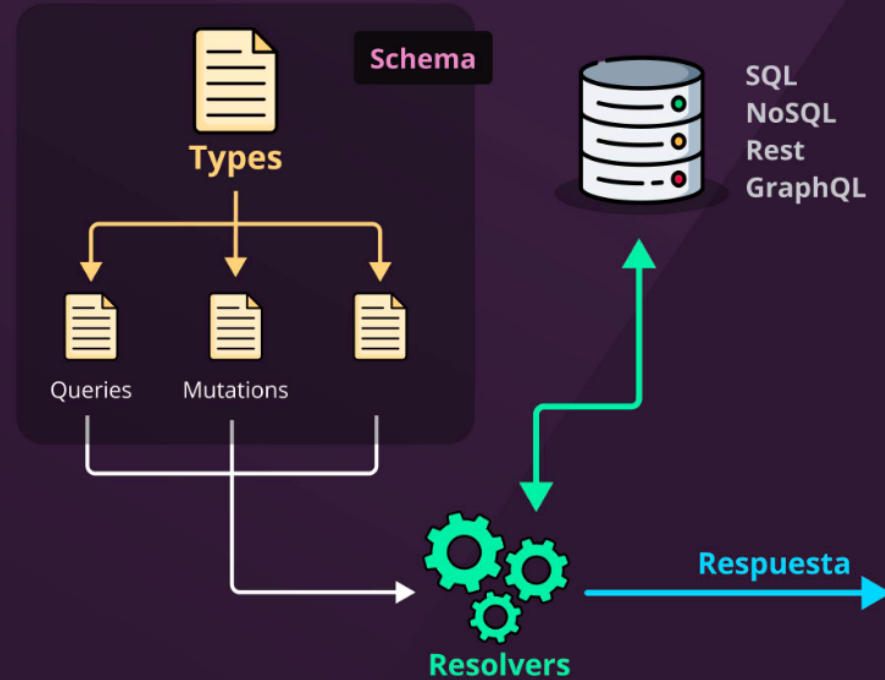
- **Jerárquico:** una consulta GraphQL en sí está estructurada jerárquicamente. La consulta tiene la misma forma que los datos que devuelve.
- **Centrado en el Producto:** se rige por los requisitos de las vistas y los ingenieros de front-end que las escriben.
- **Tipado Fuerte:** cada servidor GraphQL define un sistema de tipos específico de la aplicación. Las consultas se ejecutan dentro del contexto de ese sistema de tipos.
- **Consultas especificadas por el Cliente:** GraphQL publica las capacidades que sus clientes pueden consumir. Es el cliente especifica exactamente cómo consumirá esas capacidades publicadas.
- **Introspectivo:** El propio lenguaje GraphQL debe poder consultar el sistema de tipos de un servidor GraphQL.

¿Cómo Funciona?

¿Cómo sería el flujo de ejecución de una petición GraphQL?

```
type Query{  
  empresa(id: Int!): Empresa  
  empresas: [Empresa]  
}  
  
type Sucursal{  
  nro_sucursal: Int!,  
  direccion: String!,  
  horario_atencion: String  
}
```

¿CÓMO FUNCIONA?



```
horario_atencion: String  
): Sucursal  
}
```

LENGUAJE

Describe nuestros datos y las consultas que podemos hacer al servicio.

- Documento
- Operaciones
 - Query
 - Mutation
 - Subscription
- Conjuntos de selección, campos, alias, fragmentos
- Types:
 - SCALAR: INT, FLOAT, BOOLEAN, STRING, ID.
 - OBJECT: Es el que permite definir entidades, es uno de los más importantes para GraphQL
 - ENUMS: donde se permite elegir entre una de varias opciones.
 - INTERFACES
 - UNIONS: Permite definir diferentes posibles tipos (o interfaces) que se esperan como resultado.
- Modificadores de tipo: !, []

Ejemplo

Schema: Describe nuestros datos y las consultas que podemos hacer al servicio.

[express-graphql](#)

```
type Query{
  empresa(id: Int!): Empresa
  empresas: [Empresa]
}
type Empresa{
  id: Int,
  nombre: String,
  url: String,
  razonSocial:String!,
  rubro: Rubro!,
  ap_y_nom_apoderado:String,
  dni_apoderado:Int,
  domicilio_legal:String!,
  telefono_dl:String,
  email_dl:String,
  email_comercial:String,
  tel_comercial:String,
  empleados:Int,
  cuit:Int,
  sucursales:[Sucursal],
  balance_comercial:BalanceComercial!,
}
type Sucursal{
  nro_sucursal:Int!,
  direccion:String!,
  telefono:String!,
  localidad:String!,
  horario_atencion:String
}
type BalanceComercial {
  bc_2020:Int,
  bc_2019:Int,
  bc_2018:Int!
}
type Rubro{
  id: Int,
  nombre:String
}
type Mutation{
  SucursalAdd(
    idEmpresa: Int,
    nro_sucursal:Int!,
    direccion:String!,
    telefono:String!,
    localidad:String!,
    horario_atencion:String
  ):Sucursal
}
```

Ejemplo

Realizar consultas sobre la [API de países](#)

[Sandbox](#)

[Endpoint de Star Wars](#)

```
query{
  continents {
    name
  }
}

-----

query($continentCode: ID!){
  continents {
    name
    code
  }
  continent(code: $continentCode) {
    code
    name
  }
}

#vars
{
  "continentCode": "AF"
}
```

Conclusiones

- **Definición:** GraphQL es un lenguaje de consulta de datos, pero también un entorno de ejecución, porque permite llevar a cabo las consultas. Podemos consultar “qué queremos” y “cómo lo queremos”.
- **Ventajas:** Único Endpoint / Fuertemente Tipado / (Over/Under)fetching
- **Desventajas:** No elimina cuellos de botella / Caching
- **Comparación con REST:** GraphQL debería utilizarse cuando se requiere menor utilización de la memoria; cuando los recursos que la API maneja están estrechamente relacionados entre sí y cuando no se considere que vaya a utilizarse la data completa de los recursos.